Computers & Operations Research 168 (2024) 106685

Contents lists available at ScienceDirect

Computers and Operations Research

journal homepage: www.elsevier.com/locate/cor

Fixed set search matheuristic applied to the knapsack problem with forfeits

Raka Jovanovic [a,*], Stefan Voß [b]

[a] Qatar Environment and Energy Research Institute, Hamad bin Khalifa University, PO Box 5825, Doha, Qatar
[b] Institute of Information Systems, University of Hamburg, Von-Melle-Park 5, 20146 Hamburg, Germany

ARTICLE INFO

Keywords:
Metaheuristics
Knapsack problem with forfeits
Fixed set search
Matheuristic

ABSTRACT

This paper focuses on solving the knapsack problem with forfeits (KPF). This variation of the knapsack problem includes soft conflicts or forfeits, where forfeit pairs consist of two items and an associated penalty, which must be subtracted from the total profit if both items in the pair are chosen to be jointly in the knapsack. The proposed method combines the fixed set search (FSS) metaheuristic's learning mechanism with integer programming to solve subproblems. A new ground set of elements for the KPF is introduced to augment the information provided by the fixed set, and the method for creating fixed sets is adjusted to increase the diversity of solutions. The conducted computational experiments show that the proposed method significantly outperforms current state-of-the-art methods and finds a large number of best-known solutions for the standard test instances. Moreover, the method performs well across a wide range of parameter values. The proposed approach does not significantly exploit any specific properties of the KPF and could potentially be applied to other 0–1 problems, such as the minimum vertex cover problem or the facility location problem, without significant modifications. Additionally, the method's simplicity makes it suitable for hybridization with other metaheuristic approaches or the incorporation of specific KPF properties to improve its performance.

1. Introduction

Driven by practical applications, several traditional optimization problems have been modified to consider the potential presence of contrasting options. Typically, this situation has been represented through conflicts, which consist of pairs of decisions that cannot be taken simultaneously. Such issues are frequently represented using hard constraints. Some standard models that have been adapted to the setting with conflicts are the bin packing problem (Gendreau et al., 2004; Muritiba et al., 2010), the minimum spanning tree problem (Darmann et al., 2009), the minimum dominating set problem (Cornet and Laforest, 2018), but also the Steiner tree problem (Hill et al., 2022).

The issue of conflicts has frequently been observed in systems that are modeled using the knapsack problem (Bettinelli et al., 2017; Pferschy and Schauer, 2017) and its variations (Yamada et al., 2002; Basnet, 2018). Problems of this type have been solved using a variety of methods including scatter search (Hifi and Otmani, 2012), branch-and-bound (Bettinelli et al., 2017; Coniglio et al., 2021), and branch-and-cut (Salem et al., 2018) approaches. Another interesting variation is the quadratic knapsack problem with conflicts (Dahmani and Hifi, 2021), where a profit is generated when two compatible items are chosen, and this problem has also been addressed through heuristics.

Recently, Cerulli et al. (2020) introduced a variation of the 0/1 knapsack problem that includes soft conflicts or forfeits. In this variation, called the *Knapsack Problem with Forfeits* (KPF), each forfeit pair comprises two items and an associated penalty, which must be subtracted from the total profit if both items in the pair are chosen as part of a solution. The KPF has transversal applications, including maximizing nutrient and drug effectiveness, optimizing work shifts, managing disease within a budget, loading objects onto a truck, and selecting team members based on productivity. Cerulli et al. (2020) proposed a mathematical model and two heuristic approaches for the KPF. Capobianco et al. (2022) then proposed a hybrid metaheuristic approach that combines a genetic algorithm with the Carousel Greedy paradigm introduced by Cerrone et al. (2017). A novel variation of the KPF is the Knapsack Problem with Forfeit Sets (KPFS) (D'Ambrosio et al., 2023). In this variant, rather than to pairs, forfeit costs are associated to subsets of items. The subsets are not necessarily disjoint, and may have arbitrary cardinalities.

The focus of this paper is on finding near-optimal solutions for the KPF using a method based on the fixed set search (FSS) which is a population-based metaheuristic that adds a learning mechanism on top of the greedy randomized adaptive search procedure (GRASP) (Feo and Resende, 1995). The GRASP generates solutions using a randomized

* Corresponding author.
E-mail addresses: rjovanovic@hbku.edu.qa (R. Jovanovic), stefan.voss@uni-hamburg.de (S. Voß).

https://doi.org/10.1016/j.cor.2024.106685
Received 7 May 2023; Received in revised form 4 April 2024; Accepted 30 April 2024
Available online 3 May 2024
0305-0548/© 2024 Elsevier Ltd. All rights reserved.

greedy algorithm and applies a local search to each of them. The FSS has proven successful in solving numerous problems, such as the traveling salesman problem (Jovanovic et al., 2019), the power dominating set problem (Jovanovic and Voss, 2020), machine scheduling (Jovanovic and Voß, 2021), the minimum weighted vertex cover problem (Jovanovic and Voß, 2019), the covering location with interconnected facilities problem (Lozano-Osorio et al., 2023), the clique partitioning problem (Jovanovic et al., 2023), and bi-objective optimization problems (Jovanovic et al., 2022).

The FSS is based on the observation that high-quality solutions for a specific problem instance often share many of the same elements. Therefore, the FSS generates new solutions that contain these fixed elements, and computational resources are focused on completing the partial solution. This approach draws inspiration from earlier concepts of chunking (Voß and Gutenschwager, 1998; Woodruff, 1998), vocabulary building, and consistent chains (Sondergeld and Voß, 1999) as they relate to tabu search. Note that an explicit combination of tabu search and GRASP has been tried, too; see, e.g., Wang et al. (2013). The use of frequently occurring elements in high-quality solutions also relates to the matheuristic POPMUSIC paradigm (Taillard and Voß, 2002).

This study aims to propose an easily implementable approach for solving the KPF using a combination of the FSS and integer programming (IP). The proposed approach falls into the group of optimization methods, which are commonly referred to as matheuristics, which embed mathematical programming methods into a heuristic approach. A comprehensive overview of such methods is available in Boschetti and Maniezzo (2022). Notably, a similar method has recently proven successful in addressing the quadratic multiple knapsack problem (Galli et al., 2023). It should be noted that the kernel search (Angelelli et al., 2010; Maniezzo et al., 2021) and heuristic concentration (Rosing and ReVelle, 1997) are matheuristic approaches that involve generating multiple solutions to identify elements that are most likely to appear in high-quality solutions. The idea is to use some of these elements and solve the corresponding mathematical programming model within an iterative procedure. In the case of the kernel search, these elements do not necessarily belong to the final solution and they may even leave the kernel. More generally, the kernel search matheuristic works at the level of variable selection and lets the underlying IP solver determine their value. In a similar spirit, we may also refer to the corridor method as devised in Sniedovich and Voß (2006). In contrast, the FSS employs a mechanism that can generate various fixed sets or kernels based on elements that commonly appear in different subsets of high-quality solutions. This approach results in a more efficient global search. Although kernel search and heuristic concentration are similar to the FSS, the FSS offers a more diverse range of fixed sets or kernels.

This work presents the matheuristic FSS (MFSS) for the KPF, which introduces several new concepts beyond the basic FSS. Firstly, the effective learning mechanism of the FSS is incorporated into the matheuristic setting, which eliminates the need for a randomized greedy algorithm and local search used in the original FSS, thereby simplifying the method's implementation. Moreover, the MFSS utilizes the method for generating fixed sets to diversify the generated solutions. Finally, a method for calculating the size of the fixed set that makes it possible to effectively solve the constrained KPF is provided. Computational experiments demonstrate that the MFSS outperforms state-of-the-art methods for the KPF. We would like to mention that the MFSS has been successfully applied to the multidimensional knapsack problem and the KPFS (Jovanovic and Voß, 2024).

The paper is organized as follows. Section 2 provides the formal definition of the KPF. The next section provides details on the implementation of the MFSS for the KPF. The following section is dedicated to the presentation of the computational experiments. The paper is completed with some concluding remarks.

2. Problem definition

The knapsack problem is defined for a set V of $|V|$ items and a capacity c. Each item $i = 1..|V|$ has a positive profit value p_i and a positive weight w_i. The goal is to select a set of elements $S \subseteq V$ such that the sum of the profit values of elements in S is maximized while satisfying the constraint that the sum of the weights is less or equal to the capacity c. The Knapsack Problem with Forfeits (KPF), introduced by Cerulli et al. (2020), extends the basic problem with the presence of soft conflicts, or forfeits, between certain pairs of items. When both items from such pairs are chosen to be in the solution, a corresponding forfeit cost is deducted from the profit value. Formally, the set of all potential forfeit pairs $\hat{C}$ can be defined using the following equation.

$$\hat{C} = \{\{i,j\} \mid i \in V \wedge j \in V \wedge i \neq j\} \tag{1}$$

Eq. (1) states that the set of all potential forfeits $\hat{C}$ is a set of subsets of V with cardinality 2. Let us use the notation $C_k = \{i,j\}$, for $k = 1..|C|$, for all the forfeits in a specific set of forfeits $C \subseteq \hat{C}$. Next, let us define the positive parameters d_k, for $k = 1..|C|$, as the cost of forfeit C_k.

The integer programming model for the KPF with a set of items V and a set of forfeits $C \subseteq \hat{C}$ can be defined as follows. It uses a set of binary decision variables x_i defined for $i = 1..|V|$, which have a value of 1 if $i \in S$ is inside the knapsack and 0 otherwise. The second set of variables is v_k, defined for $k = 1..|C|$, which is used to indicate if the forfeit pair $C_k = \{i,j\} \in C$ is inside the knapsack S, or in other words, if $i,j \in S$ is satisfied.

$$\text{Maximize} \quad \sum_{i=1..|V|} p_i x_i - \sum_{k=1..|C|} d_k v_k \tag{2}$$

subject to

$$\sum_{i=1..|V|} w_i x_i \leq c \tag{3}$$

$$x_i + x_j - v_k \leq 1 \qquad \forall C_k = \{i,j\}, k = 1..|C| \tag{4}$$

$$x_i \in \{0,1\} \qquad i = 1..|V| \tag{5}$$

$$v_k \in \{0,1\} \qquad k = 1..|C| \tag{6}$$

The objective function given in (2) is equal to the sum of all profits of selected items minus the cost of all selected forfeit pairs. Constraint (3) guarantees the satisfaction of the capacity constraint. The variables x and v are bound by the constraints specified in (4). These constraints enforce that when selecting both i and j, where $C_k = \{i,j\}$, the corresponding v_k value must be equal to 1. The variable definitions are provided in (5) and (6).

3. A matheuristic based on the fixed set search

The MFSS algorithm, as in the case of the original FSS, capitalizes on the fact that many high-quality solutions for a given combinatorial optimization problem contain common elements. The MFSS generates new solutions that fix some of these elements and concentrates the computational effort on finding optimal or near-optimal solutions in the corresponding subset of the solution space. This set of common elements is referred to as the "fixed set". The goal of the MFSS is to complete the partial solution corresponding to the fixed set by "filling in the gaps." The original metaheuristic, FSS, accomplishes this by incorporating a learning mechanism into the GRASP. In this section, we present the MFSS, which extends this concept to a matheuristic setting. The MFSS is composed of various building blocks, including representing the problem solution as a subset of a ground set of elements, defining techniques for generating fixed sets, implementing the learning mechanism, and defining a method for completing a solution from a fixed set.

With the goal of having a better understanding of the MFSS algorithm, we start with giving an outline of the method. The first step is creating an initial, randomly generated population of solutions. To improve these solutions, an iterative procedure is performed that consists

of the following steps. At each iteration, first, a random base solution B is selected from the subset of high-quality solutions, having the highest values of the objective function from the previously generated ones. Then, a fixed set F is generated using this base solution and a set of k randomly selected solutions from the subset of high-quality solutions. The fixed set F is utilized to create an additionally constrained IP, for the KPF, which is then used to acquire a new solution S'. Finally, S' is added to the set of generated solutions. This process is repeated until a stopping criterion is met. In the following, we provide more details on how each step of the MFSS algorithm is implemented.

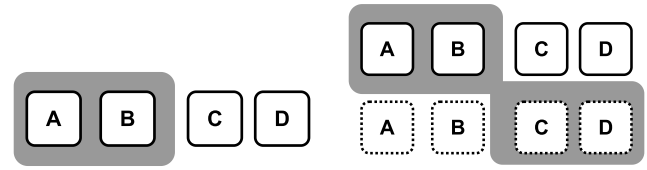
3.1. Fixed set

This section outlines a technique for generating fixed sets that can be used within the MFSS algorithm for the KPF. In order to apply the MFSS, a solution must be represented as a subset of a ground set of elements. Note that for a generic combinatorial optimization problem, the solution can be represented in form of a subset of the set of all potential elements. This set of all elements is commonly called the ground set (Nemhauser and Wolsey, 1988). Several examples, on how combinatorial optimization problems can be transformed to this type of representation can be found in Dror and Orlin (2008). Explicitly, a fixed set is a set of elements of the ground set that frequently occur in high-quality solutions. Such a fixed set must satisfy the constraint that there is a solution, preferably of high quality, of which it is a subset.[1]

For the KPF, a natural way to represent a solution is as a subset S of the set of all items V. However, for the adaptation of the FSS to a matheuristic approach, we aim to use a representation that maximizes the information that a fixed set can provide for all the items $i \in V$. To be more precise, the goal is to define the fixed set in a way that it records not only the elements that are most likely belonging to a high-quality solution but also those that are most likely not belonging to such solutions. For this purpose, we use an alternative ground set. Note that each item $i \in V$ can either be inside the knapsack or outside of it. Based on this observation, we define the ground set as $G = V \times \{\top, \bot\}$, where a pair $(i, \top)$ represents the inclusion of item i in the knapsack, while $(i, \bot)$ represents its exclusion. Thus, a solution S is a subset of the ground set G with $|S| = |V|$. That is, it has the same cardinality as the set of all items. To be more precise, for any solution S and any item $i \in V$, it must hold $|\{(i, \top), (i, \bot)\} \cap S| = 1$. Fig. 1 provides a visual comparison of the two types of ground sets. It is important to point out that there is a bijection between these two ground sets for the KPF and they are equivalent. Alternatively, we can also state that the set of all solutions $\mathcal{G}$ is defined using the following equation:

$$\mathcal{G} := \{S \mid S \subset V \times \{\top, \bot\} : |\{(i, \top), (i, \bot)\} \cap S| = 1, \forall i \in V\} \tag{7}$$

The procedure for generating multiple fixed sets with a controllable size begins with defining some terms and notations. The set of n best solutions generated in previous steps is represented by the notation $\mathcal{S}_n = \{S_1, .., S_n\}$. A base solution, which is a randomly selected solution from the best n solutions, is denoted by $B \in \mathcal{S}_n$. If a fixed set F satisfies the condition $F \subset B$, it can be used to generate a feasible solution of at least the same quality as B, e.g., simply by expanding it with the missing elements. Furthermore, F can contain any number of elements



(a) Representation of a solution using the basic ground set V (b) Representation of a solution using the extended ground set G

Fig. 1. Depiction of different ground sets for the KPF. The items inside the gray shape represent the solution. For the extended ground set, a square with full lines indicates that the item is included in the knapsack, whereas a square with dashed lines indicates the opposite.

from B. The goal is to create a fixed set F that contains frequently occurring elements in a group of high-quality solutions.

To achieve this, a function $C(e, S)$ is defined for an element $e \in G$ and a solution S as follows: if $e \in S$, $C(e, S)$ is equal to 1, otherwise it is 0. The number of times an element e occurs in the set of k randomly selected solutions out of the n best solutions, denoted by $\mathcal{S}_{kn}$, can be counted using the function:

$$O(e, \mathcal{S}_{kn}) = \sum_{S \in \mathcal{S}_{kn}} C(e, S) \tag{8}$$

It is important to emphasize that in (8) the notation $\mathcal{S}_{kn}$ represents a random set of k solutions out of the n best ones and not a specific set of solutions.

As per the desired properties of the fixed set, $F \subset B$ should contain the set of elements e with the largest values of $O(e, \mathcal{S}_{kn})$. Let us define $F = \mathrm{Fix}(B, \mathcal{S}_{kn}, \mathrm{Size})$ as the function that generates a fixed set using a base solution B and a set of solutions $\mathcal{S}_{kn}$ with a specified cardinality of the fixed set, denoted by Size. Before formally defining the function Fix, let us make some observations.

In the original FSS algorithm, the diversification of the newly generated solutions is achieved from two sources. The first one is the method for generating fixed sets which is stochastic in nature due to the random selection of the base solution B and the set of k test solutions $\mathcal{S}_{kn}$. This is also true for the MFSS. The second source is the use of the randomized greedy algorithm with a pre-selected set of elements to complete a solution from an initial fixed set. In the case of the MFSS algorithm, solutions are completed using an IP solver which is generally deterministic. Because of this, in the MFSS special attention is dedicated to the way ties are broken between elements of the base solution when selecting which of them are to be elements of the fixed set. This is done with the goal of enhancing the diversity of the generated fixed sets.

To achieve this, let us define the function $\mathrm{Freq}(B, \mathcal{S}_{kn}, t)$ for a solution B, a set of solutions $\mathcal{S}_{kn}$ which returns the set of all the elements of B that occur in the solutions in $\mathcal{S}_{kn}$ at least t times as follows.

$$\mathrm{Freq}(B, \mathcal{S}_{kn}, t) = \{e \mid e \in B \wedge O(e, \mathcal{S}_{kn}) \geq t\} \tag{9}$$

Let m be the maximum value for which the cardinality of the set $\mathrm{Freq}(B, \mathcal{S}_{kn}, m)$, satisfies $|\mathrm{Freq}(B, \mathcal{S}_{kn}, m)| \geq \mathrm{Size}$. Explicitly, m is specified using the following equations.

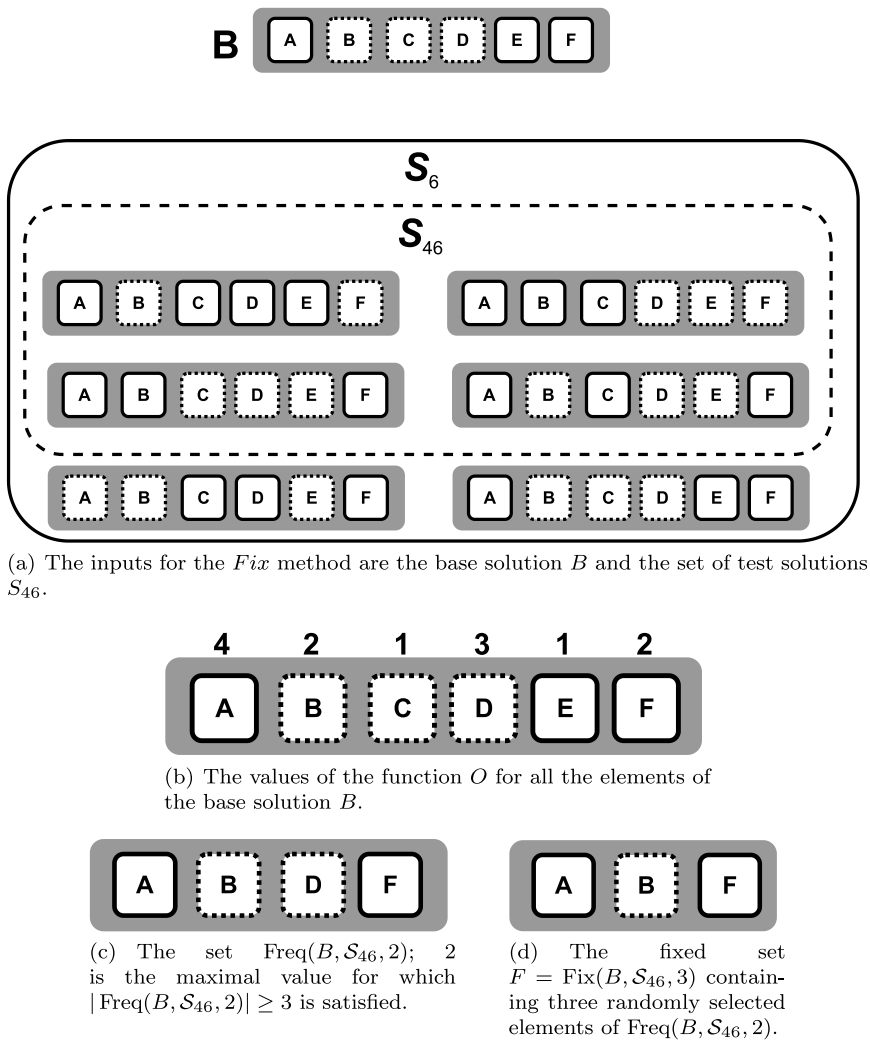
$$I = \{i \mid i \in \{1, \ldots, k\} \wedge |\mathrm{Freq}(B, \mathcal{S}_{kn}, i)| \geq \mathrm{Size}\} \tag{10}$$

$$m = \max_{i \in I} i \tag{11}$$

In Eq. (10), the set I is defined as the subset of indices i from the set $K = \{1, 2, \ldots, k\}$ where the cardinality of the set returned by $\mathrm{Freq}(B, \mathcal{S}_{kn}, i)$ is greater than or equal to the specified bound or size-constraint value Size. Eq. (11) simply states that m is equal to the maximum element of I. Consequently, a fixed set F can be acquired by selecting Size elements of $\mathrm{Freq}(B, \mathcal{S}_{kn}, m)$. The set of all potential fixed sets $\mathcal{F}$ is given in the following equation:

$$\mathcal{F} = \{W \mid W \subset \mathrm{Freq}(B, \mathcal{S}_{kn}, m) \wedge |W| = \mathrm{Size}\} \tag{12}$$

[1] For the time being, we assume that a solution is of high quality if it is among the best solutions found for the problem, e.g., among the ten percent best solutions. Setting up the fixed set of such solutions will be specified below. In literature occasionally the notion of a *reference set* is used to keep the best solutions visited during the search; see, e.g., Hanafi and Wilbaut (2008) in the context of a generalized knapsack problem. It is important to point out that the quality often can only be determined a-posteriori. Note that the notion of best solutions need not be limited to the objective function value, but can also take solution diversity into account.



(a) The inputs for the Fix method are the base solution B and the set of test solutions S_{46}.

(b) The values of the function O for all the elements of the base solution B.

(c) The set $\mathrm{Freq}(B, \mathcal{S}_{46}, 2)$; 2 is the maximal value for which $|\mathrm{Freq}(B, \mathcal{S}_{46}, 2)| \geq 3$ is satisfied.

(d) The fixed set $F = \mathrm{Fix}(B, \mathcal{S}_{46}, 3)$ containing three randomly selected elements of $\mathrm{Freq}(B, \mathcal{S}_{46}, 2)$.

Fig. 2. A depiction showcasing the process for creating fixed sets. The size of the fixed set is assumed to be Size = 3, while six best solutions and four test solutions are used. Each element of a solution is depicted using a square, with a solid or dashed line indicating whether it is inside or outside the knapsack, respectively. Solutions are represented using the elements inside a gray rectangle.

Now, the function $Fix(B, S_{kn}, \mathrm{Size})$ for generating a fixed set simply returns a random element F of the set of all fixed sets $\mathcal{F}$. It is important to note that the elements $e \in F$ of the fixed set acquired using the function $Fix(B, S_{kn}, \mathrm{Size})$ do not necessarily have the largest value of $O(e, \mathcal{S}_{kn})$. A visual representation of the method for generating fixed sets can be seen in Fig. 2.

3.2. Utilization of the integer program in the matheuristic fixed set search

The proposed matheuristic approach aims to capitalize on the following behavior of IP solvers that has been empirically observed. For many problems, for instances up to a certain size, the solver manages to find optimal solutions within a short computational time. Even more, for such instances, standard IP solvers may acquire near-optimal (or even optimal) solutions quickly, and then eventually spend the majority of the time finding an optimal solution and proving its optimality.

3.2.1. Integer program with fixed variables

To address this last-mentioned challenge, the proposed approach utilizes fixed sets to create IPs that can be *efficiently solved* using standard solvers. The term *efficiently solved* is used in the context of applying an IP solver within the MFSS algorithm. In it, the IP solver is used to complete a large number of partial solutions corresponding to fixed sets. The idea is that these solutions, completed by the IP solver, are of high quality, preferably near optimal, for the related subproblem (restricted solution space). In practice, this means that the solver should be able to find such a solution for the IP within a short time period, since many such subproblems need to be solved. In this context, an IP can be *efficiently solved* if a high-quality solution can be acquired using the solver within a short computational time.

Here the term "short computational time" is largely related to the machine on which the proposed method is executed. In general, it has been observed that the proposed MFSS frequently starts stagnating after the IP solver completes 5000 to 10 000 partial solutions. For example, to maximize the performance of the MFSS with a computation time limit of 600 s, it should complete approximately that many partial solutions, consequently the computational time limit given to the IP solver should be around 0.05 to 0.1 s. The quality of solutions that should be found by the IP solver is highly problem dependent, but some guidelines for selecting the size of the subproblems being solved by the IP solver will be described in Section 4.3.

A straightforward way to decrease the computational cost of solving an IP is to fix the values of certain decision variables. This can be accomplished naturally using fixed sets by adding a set of constraints to the IP. Specifically, we add Constraints (13) and (14) to the model

(2)–(6):

$$x_i = 0 \qquad i \in \{j \mid (j, \bot) \in F\} \tag{13}$$

$$x_i = 1 \qquad i \in \{j \mid (j, \top) \in F\} \tag{14}$$

Constraint (13) ensures that any element of the fixed set F of the form $(i, \bot)$ represents an item i that is not inside the knapsack. Similarly, for an element $(i, \top)$, i is in the knapsack.

We define $\mathrm{IPS}(F, \tau)$ as a function that returns a best-quality solution that the IP solver acquires for the IP defined by (2)–(6), (13), and (14) within a computational time limit τ. Note that, in the case the fixed set F is equal to the base solution B, the only solution of the corresponding IP Solver is the base solution; because of this it is preferable that F is a proper subset of B.

3.2.2. Use of an incumbent solution

In the practical application of IP solvers, providing a high-quality incumbent solution S as a "warm start" can often yield a significant decrease in computational time and, consequently, in the quality of found solutions within a given time limit. Some examples of the positive effect of using high-quality incumbent solutions can be found in the works of Pour et al. (2018), Chen et al. (2020), de Abreu and Fuchigami (2022) and Caserta and Voß (2013). Providing an initial solution from the start of the branch-and-cut algorithm can exclude sections of the search space, potentially leading to smaller branch-and-cut trees. On the other hand, the presence of an incumbent solution can permit the IP solver to employ heuristics that rely on it, such as the relaxation-induced neighborhood search (RINS heuristic; Danna et al. (2005)) or solution refinement techniques.

In the proposed method, the use of an incumbent solution is leveraged to improve the performance of the IP solver. Let us introduce the function $\mathrm{IPS}(F, \tau, S)$, which extends the IPS function by incorporating an initial incumbent solution S. Let $\mathrm{Ob}(S)$ denote the objective value of solution S. Additionally, let us define $\mathrm{BestFit}(S, F)$ as a function that returns a highest-quality solution $S \in \mathcal{S}$, where $\mathcal{S}$ is a set of solutions, that can be obtained by expanding the fixed set F as follows:

$$\mathrm{Fit}(S, F) = \{S \mid S \in \mathcal{S} \wedge F \subseteq S\} \tag{15}$$

$$\mathrm{BestFit}(S, F) = \mathrm{argmax}_{S \in \mathrm{Fit}(S,F)} \mathrm{Ob}(S) \tag{16}$$

Eq. (15) defines $\mathrm{Fit}(S, F)$ as the set of all solutions $S \in \mathcal{S}$ for which $F \subseteq S$. Then Eq. (16) returns a solution that achieves the highest objective value. Note that there can be more than one solution having an objective equal to this value; in such a case a random one of them is returned.

3.2.3. Fixed set size

In the proposed MFSS, an important part of IP usage is the selection of the size of the fixed set. From our initial experiments, it has been observed that the computational cost of the IPS depends on the sum of the number of free variables of type x_i related to the inclusion of items in the knapsack and the number of free variables of type v_k related to the forfeits. The usage of the fixed set directly fixes the values of variables x_i, such that $(i, \top)$ or $(i, \bot)$ is an element of the fixed set. On the other hand, due to the fact that the coefficients d_k are positive and the objective is a maximization one, some of the variables v_k are strictly specified by fixing the value of a variable x_i. To be exact, a variable v_k related to the forfeit pair $C_k = \{i, j\}$ can be eliminated if $(i, \bot)$, $(i, \top)$, $(j, \bot)$ or $(j, \top)$ are within the fixed set F. The rules for eliminating the variables v_k are given as follows.

$$\text{If } (i, \bot) \in F \qquad x_i = 0 \implies v_k = 0 \tag{17}$$

$$\text{If } (i, \top) \in F \qquad x_i = 1 \implies v_k = x_j \tag{18}$$

$$\text{If } (j, \bot) \in F \qquad x_j = 0 \implies v_k = 0 \tag{19}$$

$$\text{If } (j, \top) \in F \qquad x_j = 1 \implies v_k = x_i \tag{20}$$

Eq. (17) states that in the case constraint $x_i = 0$ is added ($(i, \bot) \in F$), the value of variable v_k is equal to zero due to the fact it is not possible for the conflict $C_k = \{i, j\}$ to occur. Eq. (18) states that in the case constraint $x_i = 1$ is added ($(i, \top) \in F$), the variable v_k can be eliminated since the conflict C_k can only occur if item j is selected or, equivalently, $v_k = x_j$. Eqs. (19), (20) provide analogous rules for the inclusion of $(j, \bot)$ or $(j, \top)$ in the fixed set.

It is evident that for a known fixed set F it is trivial to calculate the total number of free variables of type x_i and v_k. The problem is that for a given size of the fixed set it is not known which specific fixed sets will be generated. Because of this, an estimation of the number of free variables for a fixed set size $|F|$ is used. The idea is to estimate the number of free variables by only using the cardinality of the corresponding sets, $|V|$, $|C|$ and $|F|$, without considering the specific elements of these sets.

The number of free variables related to items is equal to the total number of variables $|V|$ minus the number of fixed ones $|F|$. The number of free variables related to forfeit pairs is estimated using a proportion with the case when all possible conflict pairs are present. For the sake of completeness, the related calculation is provided in the following text. Let us assume that all the conflict pairs are present. In this case the number of variables that are removed for a fixed set with $|F|$ elements can be calculated using the following function.

$$R(|F|, |V|) = |F|(|V| - |F|) + \frac{|F|(|F| - 1)}{2} \tag{21}$$

In (21), function R represents the number of combinations of two elements that have at least one element selected from $|F|$ out of $|V|$ elements. The first term, i.e. $|F|(|V| - |F|)$, represents the number of combinations of two elements where one element is from the selected $|F|$ elements and the other element is from the remaining $|V| - |F|$ items. The second term, $\frac{|F|(|F|-1)}{2}$, represents the number of combinations of two elements from the selected $|F|$ items.

Using $R(|F|, |V|)$, an estimate of the number of free variables for a problem instance with $|V|$ items, $|C|$ forfeits and a fixed set with $|F|$ elements can be made using the following function.

$$\Phi(|F|, |V|, |C|) = |V| - |F| + |C| \left(1 - \frac{R(|F|, |V|)}{|\hat{C}|}\right) \tag{22}$$

Eq. (22) states that $\Phi(|F|, |V|, |C|)$ is equal to the sum of the number of free variables x_i related to items $(|V| - |F|)$ and free variables v_k related to forfeit pairs. The latter is equal to the number of forfeit pairs $|C|$ scaled by the portion of variables that are free for the case that the forfeit set is the complete one $\hat{C}$. Note that $\hat{C}$ has $\frac{|V|(|V|-1)}{2}$ elements.

Let us assume that the IPS can be efficiently solved for Γ free variables. How large should the fixed set be? It is evident that a larger part of the solution space is explored for a smaller size of the fixed set, and consequently the local search is more efficient. Because of this, the minimal size $|F|$ of the fixed set, for which $\Phi(|F|, |V|, |C|) \leq \Gamma$, should be selected.

3.3. Algorithm overview

In this section, an overview of the MFSS algorithm is provided. The proposed method can be best understood by observing the pseudocode presented in Alg. 1. In the following text, each step of the algorithm is explained.

The first part of the pseudocode initializes the values of several variables that are important for the method. To be specific, these are the method stagnation counter Stag, the value of the best-found solution S_{best} and the current time limit for solving the IPS τ. Note that the value of τ is set based on the input parameter τ_{init}.

The next step is dedicated to setting the size of the fixed set, denoted as Size, which corresponds to the size of a subproblem that can be

Algorithm 1 Pseudocode for MFSS

Input: Initial population size N_{pop}, the base and test solutions are selected from the best n solutions, subproblem size Γ, initial computational time for the IP τ_{init}, $k_{\min}$ and $k_{\max}$ specify the range from which the number of test solutions is used in $\mathcal{S}_{kn}$
Output: The best-found solution S_{best}
function $MFSS(N_{\text{pop}}, n, \text{Size}_{\max}, \tau_{\text{init}}, k_{\min}, k_{\max})$
  $\tau \leftarrow \tau_{\text{init}}$
  $\text{Stag} \leftarrow 0$
  $S_{\text{best}} \leftarrow 0$
  Calculate Size based on Γ as described in Section 3.2.3
  Generate initial population $\mathcal{P}$
  while Not Time Limit Reached **do**
    Select random $k \in [k_{\min}, k_{\max}]$
    Select random $B \in \mathcal{S}_n$
    Generate random $\mathcal{S}_{kn}$
    $F \leftarrow \text{Fix}(B, \mathcal{S}_{kn}, \text{Size})$
    $S_{\text{start}} \leftarrow \text{BestFit}(\mathcal{P}, F)$
    $S \leftarrow \text{IPS}(F, \tau, S_{\text{start}})$
    if $\text{Ob}(S) < \text{Ob}(S_{\text{best}})$ **then**
      $S_{\text{best}} \leftarrow S$
    end if
    if $S \notin \mathcal{P} \wedge \text{Ob}(S) \geq \text{Ob}(S_n)$ **then** ▷ S_n is the n-th best solution
      $\text{Stag} = 0$
    else
      $\text{Stag} = \text{Stag} + 1$
    end if
    if $\text{Stag} \geq \text{StagMax}$ **then**
      $\text{Stag} = 0$
      $\tau \leftarrow 2\tau$
    end if
    $\mathcal{P} = \mathcal{P} \cup \{S\}$
  end while
end function

solved efficiently using the IPS. The size is determined as the minimal size of the fixed set for which the number of decision variables in the IPS is less than or equal to Γ, where Γ is an input parameter.

Next, an initial population of N_{pop} of random feasible solutions is generated. The proposed method uses the following iterative procedure to generate a random solution. It begins with an empty solution $S = \emptyset$, an initial partial solution, and then iteratively adds items $t \in V$ to S in a random order, as long as the capacity constraint is not violated. The notation $\mathcal{P} = \{S_1, \ldots, S_{N_{\text{pop}}}\}$ is used for the population of solutions within the Algorithm 1. Note that in the pseudocode, the notation $\mathcal{S}_n$ is used for the n best solutions of the population of solutions $\mathcal{P}$ generated in the previous steps of the algorithm.

After the initialization steps, the next part of the pseudocode corresponds to the main loop which implements the learning mechanism of the MFSS. Its main goal is to iteratively add solutions of increasing quality to the population of solutions $\mathcal{P}$. In it, firstly, an integer value k is randomly selected to determine the size of the set $\mathcal{S}_{kn}$, with the value of k changing in subsequent iterations to increase the diversity of the generated fixed sets. The random base solution B and the set $\mathcal{S}_{kn}$ are chosen from the set of the n best solutions.

Afterwards, the function $\text{Fix}(B, \mathcal{S}_{kn}, \text{Size})$ is applied to generate a fixed set F. The initial incumbent solution S_{start} is then selected as the best solution in the population $\mathcal{P}$ that fits the fixed set F. The next step involves using the function $\text{IPS}(F, \tau, S_{\text{start}})$ with a time limit τ and the initial incumbent solution S_{start} to obtain a new solution S. The solution S is then evaluated to determine if it is the new best solution. Next, it is checked if S is a new solution ($S \notin \mathcal{P}$) and whether it is among the n best generated solutions.

If S is not added to the set $\mathcal{S}_n$, the stagnation counter is increased by 1; otherwise, it is set to 0. The procedure then checks if stagnation has occurred, and if so, the allowed computational time τ for IPS is doubled. Finally, the new solution S is added to the population of solutions $\mathcal{P}$, and the process is repeated until the time limit is reached.

Note that an essential part of the MFSS is the increase of the time limit τ of the IPS, which is explained in the following text. When implementing the learning mechanism in the iterative procedure, there are several factors to consider. One important aspect is the computational cost of the IPS, which can potentially be quite high. Specifically, the quality of the fixed set being used can have a significant impact on the effectiveness of the IPS. For low-quality fixed sets, even allowing the IPS to run for a long time may not yield high-quality solutions. In contrast, for high-quality fixed sets, longer execution times for the IPS are more likely to result in high-quality solutions, since they explore a part of the solution space that is more likely to contain such solutions.

As the algorithm generates more solutions, the quality of the fixed sets also improves, which means that longer execution times for the IPS become more appropriate as the algorithm progresses. To account for this, the proposed method allows shorter computational times for the IPS during the early iterations of the algorithm and gradually increases the allowed time as the algorithm progresses. Specifically, the algorithm starts with an initial allowed computational time τ for the IPS and increases it when the algorithm begins to stagnate. When no new solution that is among the n best solutions has been generated in the last StagMax iterations, the algorithm is considered stagnant, and the allowed computational time is increased to help the algorithm explore more of the solution space.

4. Results

This section presents the outcomes of the computational experiments that were conducted. The primary aim is to assess the efficiency of the proposed MFSS, and this is carried out by comparing it with state-of-the-art metaheuristic approaches. The secondary goal is to examine the impact of method parameters. The final aim is to evaluate the diversity of generated fixed sets during the execution of the MFSS since it can provide some insight in the ability of the method to avoid being trapped in locally optimal solutions. The implementation of the MFSS method was performed using ILOG CPLEX 20.1.0 within Concert Technology for C#.NET. The computational experiments were conducted on a personal computer operating on Windows 10, equipped with an Intel(R)Xeon(R) Gold 6244 CPU @3.60 GHz, and 128 GB of memory.

All the test instances in the comparison employed an identical set of MFSS parameters. The initial population size N_{pop} was set to 200, and the maximal number of free variables in the subproblem Γ is set to 1000. It is important to point out that the chosen value for Γ is found empirically. To be more precise, it was the largest size of the subproblem for which the underlying IPS could find expected near-optimal solutions for a low computational cost. The initial maximal computational time allotted for the IPS was 0.1 s. Note that in our implementation of the MFSS, the computational time limit τ for the IPS is used to set the value of the $TimeLimit$ parameter of the CPLEX solver. It is interesting to mention that we have empirically observed that for the value of 0.1 of this CPLEX parameter, the solver generally visited zero non-root branch-and-cut nodes. For selecting the base solution and $\mathcal{S}_{kn}$, the number of n best solutions considered was 200. The value of the parameter k used for generating $\mathcal{S}_{kn}$ was a random integer value selected from the interval $[5, 9]$. If there had been no changes to the set of the n best solutions $\mathcal{S}_n$ in the last $\text{StagMax} = 50$ iterations, the algorithm was deemed stagnant. The MFSS was allowed a maximum computation time of 600 s.

The source code of the method, test instances and detailed results of the performed computational experiments can be found at https://github.com/rakabog/KPF .

4.1. Test instances

The computational experiments are carried out on the instances that have been initially introduced by Cerulli et al. (2020) and later expanded by Capobianco et al. (2022). In the following text, a short description of the instances is provided. The data set comprises of three groups, each containing 40 instances, defined as follows:

- **Original (O) instances**: In these instances the number of items $|V|$ belongs to the set $\{500, 700, 800, 1000\}$, and the number of forfeit pairs $|C|$ is equal to $6|V|$, each composed of a random pair of items. The budget c is equal to $3|V|$. The item weights, profits, and forfeit costs are randomly chosen integers in the intervals $[3, \dots, 20]$, $[5, \dots, 25]$, and $[2, \dots, 15]$, respectively. For each value of $|V|$, ten different instances were generated.
- **Large Knapsack (LK) instances**: These instances are the same as the O ones, with a larger budget equal to $5|V|$.
- **More Forfeits (MF) instances**: These instances have the same budget and the same items as the O ones. Each item has the same associated profit and weight. For each instance, $8|V|$ new random forfeit pairs are generated. Again, costs are chosen randomly in the interval $[2, \dots, 15]$.

We would like to mention that we have observed that some of the instances, in this set of test instances, have multiple forfeits with the same pair of items. In such case, the sum of all forfeit costs is used.

4.2. Comparison to other methods

To evaluate the efficacy of the proposed algorithm, a comparison is done to the published state-of-the-art methods. The greedy algorithm (GR) proposed by Cerulli et al. (2020) is used as a base for comparison. The second method is the extension of the basic greedy algorithm with the carousel greedy paradigm (CG) also proposed by Cerulli et al. (2020). The hybrid metaheuristic proposed by Capobianco et al. (2022) called GA–CG Forfeits which combines the strengths of the genetic and carousel greedy paradigms is also used for comparison. Finally, in the comparison we also include the results acquired using IPs, provided by Capobianco et al. (2022), within a time limit of three hours, which are the best-known solutions (BKS). To verify the effectiveness of the MFSS, it is compared to the use of CPLEX for solving the IP model. To be specific, the quality of solutions acquired by CPLEX within a time limit of 600 s is included in the comparison.

The computational experiments are performed on the instances described in the previous subsection. Since the MFSS is a stochastic method, ten independent runs are performed for each problem instance. For the MFSS the best-found solution and average solution quality are observed. In case of the other methods, the results of the competing methods only included the best-found solutions. The results of the corresponding computational experiments can be seen in Tables 1–3 for instances of type O, LK and MF, respectively. In addition, in Table 4, the average execution time over all the instances of one type and size are provided for each of the competing methods. With the intention of having a better understanding of the convergence speed of the MFSS the average time-to-best solution is also provided in this table. For the other methods the computational time is scaled using the factor of 0.44 based on the used CPUs and the information provided by PassMark (2022). It should be noted that, based on our understanding, in case of the GA–CG the total computational time is provided in Capobianco et al. (2022). The results obtained by the IP, provided by Capobianco et al. (2022), are currently the best-known ones for the test instances.

The first thing that can be observed from the results in Tables 1–3 is that the use of the CG extension of the GR provides a significant improvement for all the test instances for all the problem types. The use of population-based approaches, GA–CG and MFSS, has a significant advantage compared to the single solution methods (GR and CG). The MFSS has a significant advantage compared to GA–CG and manages to find better solutions for all of the 120 test instances of different types and sizes. It is important to point out that the average solution quality over ten runs of the MFSS, for all the instances, has a higher value than the ones acquired by the GA–CG. The comparison between the solutions acquired by CPLEX and the MFSS with the same time limit of 600 s illustrates the effectiveness of the proposed matheuristic approach. The average solution quality for the ten independent runs of the MFSS has a higher value than the solution acquired by CPLEX for all but four instances. For these four instances both the solutions acquired by CPLEX and the average solution values of the MFSS are equal to the proven optimal values.

The two basic methods GR and CG have a significantly lower – 100 times – computational time than the two population-based methods. This is an expected result and is commonly observed when comparing greedy algorithms to population-based ones that generate a large number of solutions. When comparing the computational time of the GA–CG and MFSS, several observations can be made, even though the time needed to acquire the best solution for GA–CG is not provided in the related paper. The total computational time of the GA–CG is lower than the total computational time of MFSS (600 s) for instances up to 800 items. In the case of instances with 1000 items the computational time of MFSS becomes lower. It should be noted that the time MFSS

Table 1
Comparison of the state-of-the-art methods for the KPF to the MFSS for problem instances of type O. The proven optimal solutions are underlined for BKS. The new best-known solutions are marked in bold text. The abbreviation BKS is used for best-known solution.

Size	ID	Best solution						Avg. solutions
		BKS	CPLEX	GR	CG	GA–CG	MFSS	MFSS
500	1	2626	2624	2309	2510	2568	2626	2626.0
500	2	2660	2657	2428	2556	2621	2660	2660.0
500	3	2516	2500	2335	2400	2478	2516	2516.0
500	4	2556	2556	2327	2441	2515	2556	2556.0
500	5	2625	2620	2328	2502	2582	2625	2625.0
500	6	2615	2615	2309	2500	2557	2615	2615.0
500	7	2627	2614	2316	2470	2602	2627	2626.8
500	8	2556	2556	2282	2471	2522	2556	2556.0
500	9	2613	2613	2337	2524	2572	2613	2613.0
500	10	2558	2539	2346	2439	2537	2558	2557.7
700	1	3589	3584	3218	3448	3511	3589	3589.0
700	2	3422	3414	2865	3253	3359	**3424**	3424.0
700	3	3679	3678	3374	3449	3634	3679	3678.7
700	4	3664	3660	3380	3512	3605	3664	3662.6
700	5	3647	3642	3332	3457	3619	3647	3646.4
700	6	3596	3587	3330	3447	3553	3596	3596.0
700	7	3542	3531	3086	3319	3446	3542	3542.0
700	8	3619	3593	3183	3389	3545	3619	3617.6
700	9	3553	3527	3040	3363	3487	3553	3551.5
700	10	3652	3646	3167	3462	3594	3652	3650.9
800	1	4184	4168	3877	4024	4125	4184	4180.6
800	2	4065	4046	3649	3827	4006	4065	4063.2
800	3	4102	4075	3684	3886	4018	**4107**	4104.7
800	4	4051	4006	3633	3850	3960	**4056**	4052.4
800	5	4085	4052	3622	3900	4041	**4088**	4087.7
800	6	4249	4242	3790	4084	4184	4249	4249.0
800	7	4121	4087	3682	3897	4021	**4126**	4125.6
800	8	4063	4044	3636	3859	4019	4063	4062.1
800	9	4080	4069	3703	3853	4017	4080	4078.2
800	10	4124	4122	3794	4050	4074	**4131**	4130.5
1000	1	4927	4910	4455	4655	4834	**4940**	4939.6
1000	2	4966	4923	4562	4756	4893	**4994**	4988.0
1000	3	5171	5122	4724	4897	5070	**5178**	5175.7
1000	4	5141	5121	4602	4916	5065	**5144**	5135.2
1000	5	5134	5108	4693	4935	5049	**5136**	5135.1
1000	6	5082	5063	4606	4858	4951	**5083**	5077.9
1000	7	5100	5062	4482	4876	5033	**5121**	5113.5
1000	8	5178	5169	4780	4916	5071	**5191**	5186.1
1000	9	5108	5091	4536	4890	5011	**5115**	5106.6
1000	10	5178	5141	4724	4998	5080	**5181**	5179.1

Table 2
Comparison of the state-of-the-art methods for the KPF to the MFSS for problem instances of type LK. The new best-known solutions are marked in bold text. The abbreviation BKS is used for best-known solution.

Size	ID	Best solution						Avg. solutions
		BKS	CPLEX	GR	CG	GA–CG	MFSS	MFSS
500	1	2712	2703	2309	2528	2649	**2719**	2718.6
500	2	2729	2668	2431	2580	2666	**2739**	2739.0
500	3	2639	2560	2339	2436	2587	2639	2639.0
500	4	2665	2640	2329	2474	2600	2665	2665.0
500	5	2686	2657	2328	2509	2615	**2689**	2689.0
500	6	2746	2743	2333	2542	2707	**2754**	2754.0
500	7	2689	2662	2316	2484	2659	**2706**	2700.6
500	8	2681	2659	2290	2574	2644	2681	2680.1
500	9	2652	2611	2337	2524	2615	2652	2652.0
500	10	2665	2658	2365	2524	2619	**2675**	2673.3
700	1	3757	3702	3225	3487	3678	**3761**	3756.5
700	2	3611	3560	2865	3288	3535	**3612**	3606.4
700	3	3824	3772	3404	3510	3799	**3837**	3834.2
700	4	3835	3797	3397	3550	3749	**3840**	3836.7
700	5	3823	3694	3353	3585	3750	**3842**	3838.9
700	6	3707	3659	3337	3526	3651	**3719**	3719.0
700	7	3676	3539	3086	3326	3624	**3685**	3682.3
700	8	3762	3729	3199	3538	3735	**3793**	3790.6
700	9	3651	3582	3040	3393	3603	**3655**	3651.3
700	10	3832	3760	3175	3499	3736	3832	3830.8
800	1	4298	4205	3893	4084	4213	**4307**	4303.6
800	2	4201	4142	3653	3875	4129	**4219**	4216.3
800	3	4251	4105	3689	3912	4128	**4257**	4255.6
800	4	4209	4074	3645	3924	4146	**4217**	4214.5
800	5	4176	4061	3626	3971	4145	**4231**	4229.9
800	6	4417	4343	3794	4137	4343	**4422**	4416.8
800	7	4284	4259	3683	3920	4175	**4300**	4294.4
800	8	4144	4037	3636	3913	4120	**4158**	4154.6
800	9	4271	4160	3729	3976	4225	**4306**	4301.5
800	10	4277	4130	3809	4078	4192	**4285**	4280.3
1000	1	5147	5019	4460	4690	5037	**5184**	5175.8
1000	2	5156	4993	4565	4841	5025	**5161**	5157.0
1000	3	5340	5225	4724	4940	5281	**5386**	5376.7
1000	4	5421	5255	4616	5099	5323	**5431**	5428.8
1000	5	5254	5208	4709	5018	5209	**5307**	5301.3
1000	6	5345	5205	4643	4964	5234	**5356**	5344.8
1000	7	5244	5144	4490	4971	5139	**5268**	5257.3
1000	8	5323	5135	4792	5020	5245	**5390**	5375.1
1000	9	5295	5140	4548	5031	5225	**5343**	5334.9
1000	10	5362	5198	4732	5119	5318	**5396**	5392.9

Table 3
Comparison of the state-of-the-art methods for the KPF to the MFSS for problem instances of type MF. The new best-known solutions are marked in bold text. The abbreviation BKS is used for best-known solution.

Size	ID	Best solution						Avg. solutions
		BKS	CPLEX	GR	CG	GA–CG	MFSS	MFSS
500	1	2368	2347	1937	2223	2305	2368	2367.8
500	2	2310	2270	1906	2027	2300	**2318**	2316.6
500	3	2284	2237	1830	2108	2229	2284	2282.4
500	4	2259	2244	1973	2098	2228	**2270**	2268.4
500	5	2321	2270	2022	2199	2272	2321	2321.0
500	6	2316	2291	2010	2230	2283	**2319**	2316.7
500	7	2288	2246	1977	2129	2207	**2290**	2290.0
500	8	2201	2129	1876	2052	2161	**2215**	2210.2
500	9	2259	2199	1967	2034	2219	**2263**	2262.9
500	10	2305	2286	1863	2069	2285	**2310**	2308.8
700	1	3127	3023	2622	2915	3050	**3128**	3124.4
700	2	3038	2926	2568	2775	2966	**3052**	3045.4
700	3	3197	3148	2763	3000	3162	**3225**	3216.1
700	4	3233	3184	2794	2994	3176	**3247**	3245.1
700	5	3238	3120	2769	3035	3134	**3246**	3239.7
700	6	3129	3091	2673	2901	3095	**3133**	3126.8
700	7	3015	2924	2493	2668	2948	**3048**	3032.0
700	8	3166	3097	2679	2924	3096	**3176**	3175.6
700	9	3186	3169	2759	3017	3146	**3216**	3202.6
700	10	3203	3053	2637	2940	3154	**3214**	3208.8
800	1	3691	3484	3073	3428	3639	**3705**	3703.6
800	2	3711	3630	3239	3489	3647	**3727**	3726.3
800	3	3605	3478	3117	3398	3566	**3672**	3665.3
800	4	3490	3338	3008	3203	3391	**3519**	3506.6
800	5	3741	3659	3274	3483	3704	**3753**	3752.3
800	6	3772	3698	3375	3502	3697	**3775**	3774.9
800	7	3683	3518	3267	3442	3611	**3690**	3678.5
800	8	3575	3539	3167	3334	3524	**3612**	3587.8
800	9	3593	3505	3057	3340	3521	**3604**	3596.9
800	10	3633	3505	3067	3336	3531	**3646**	3638.1
1000	1	4450	4258	3875	4112	4393	**4469**	4464.7
1000	2	4408	4246	3805	4133	4337	**4474**	4469.4
1000	3	4577	4436	3863	4268	4572	**4636**	4622.8
1000	4	4564	4185	3931	4122	4468	**4591**	4587.1
1000	5	4468	4237	3901	4210	4399	**4531**	4514.9
1000	6	4569	4488	3833	4258	4512	**4599**	4596.4
1000	7	4578	4495	3971	4321	4485	**4622**	4604.5
1000	8	4486	4119	3818	4167	4421	**4557**	4549.7
1000	9	4631	4516	3973	4430	4570	**4649**	4646.3
1000	10	4660	4245	3977	4345	4559	**4672**	4662.3

needs to find the best solution, except for the smallest instances, is consistently lower than the total computational time of the GA–CG. Another important observation is that the MFSS has a good scaling of the computational time which is close to linear in relation to the instance size which is not the case for GA–CG.

The effectiveness of the MFSS is best illustrated when the quality of the generated solutions is compared to the previously best-known ones acquired by the IP. Firstly, the MFSS manages to acquire all proven optimal solutions (39 out of 120 test instances). The MFSS always acquires a solution that is at least as good as the previously best-known one. In addition it has found 88 new best solutions.

4.3. Method parameters

In this subsection, we delve into the impact of various parameters on the performance of the MFSS algorithm. Like its counterpart, the FSS algorithm (Jovanovic et al., 2019; Jovanovic and Voß, 2019), the MFSS algorithm boasts exceptional robustness, delivering good results across a broad range of parameter values. It is essential to note that the execution time of the IP Solver is hardware-dependent, which renders some of the parameters tied to it. Therefore, our focus is not on specific values but on their effect on the algorithm.

Table 4
Comparison of the execution times.

Size	Type	Average total time [s]				Average time-to-best [s]
		GR	CG	GA–CG	MFSS	MFSS
500	O	0.23	0.57	72.11	600.00	103.75
700	O	0.61	1.58	222.74	600.00	141.27
800	O	0.95	2.45	351.21	600.00	164.85
1000	O	2.11	5.48	698.10	600.00	204.22
500	LK	0.22	0.59	98.31	600.00	93.46
700	LK	0.62	1.60	301.11	600.00	160.41
800	LK	0.95	2.47	481.12	600.00	162.47
1000	LK	2.15	5.57	923.37	600.00	182.06
500	MF	0.27	0.71	111.74	600.00	217.81
700	MF	0.79	2.14	325.98	600.00	270.50
800	MF	1.25	3.39	491.01	600.00	273.74
1000	MF	2.49	6.80	926.76	600.00	309.00

The algorithm's first step involves generating the initial population of solutions, and Sarhani et al. (2023) provide a comprehensive explanation of this process in metaheuristics. To ensure best possible learning, it is critical to generate at least n solutions, which the algorithm uses. Hence, N_{pop} should be greater or equal than n. Random methods are typically used to generate solutions, which can produce

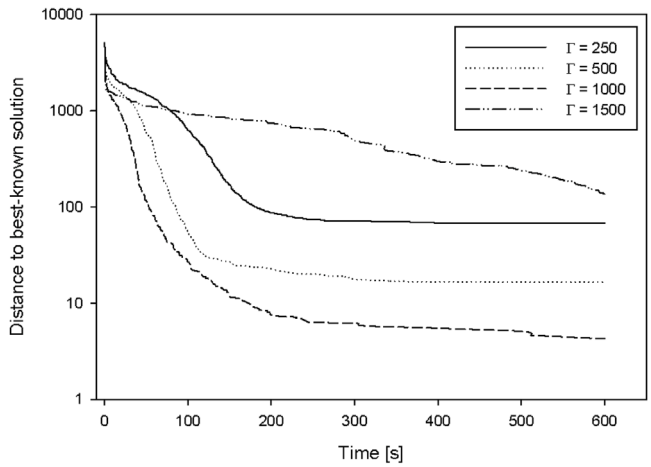


Fig. 3. Illustration of the convergence speed for different values of Γ for the instance with 1000 items of type MF and $ID = 1$. The graphs represent the average distance to the best-known solution over ten runs at different time periods.

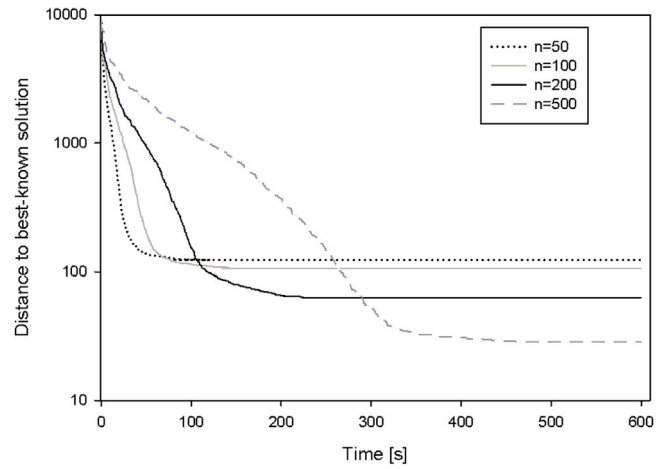


Fig. 4. Illustration of the convergence speed for different values of n for the instance with 1000 items of type LK and $ID = 1$. The MFSS solves subproblems with $\Gamma = 250$. The graphs represent the average distance to the best-known solution over ten runs at different time periods.

low-quality feasible solutions. Consequently, using N_{pop} values higher than n does not offer significant advantages. However, suppose more advanced methods that incorporate local search are used to generate initial solutions. In that case, higher values of N_{pop} are advisable, given the potential to acquire more high-quality solutions. This is because the IPS is computationally expensive, and it is preferable to apply it only to high-quality solutions.

The parameters $k_{\min}$ and $k_{\max}$ serve to diversify the generation of new solutions by randomizing the size of set S_{kn}. A lower value of k produces fixed sets with greater randomness. When there are fewer test solutions, more elements $e \in B$ have the same $O(e, S_{kn})$ value, so a larger portion of them are randomly removed from $\mathrm{Freq}(B, S_{kn}, m)$. Conversely, a higher k value makes the fixed set generation more deterministic by reducing the number of such elements, thereby decreasing the difference between Size and $|\mathrm{Freq}(B, S_{kn}, m)|$.

The initial computational time τ_{init} of the IPS is found empirically and is highly dependent on the used hardware. Ideally, it should be the lowest value that still has a high likelihood of improving the solutions in the initial population. In contrast, StagMax increases the IPS's computational time and is effective for a wide range of values. However, once the solution space is sufficiently explored beyond a certain minimum value for StagMax, increasing it further only incurs additional costs without providing extra benefits. Our initial tests suggest that StagMax = 50 was best for the hardware and problem we used.

The performance of the MFSS, in terms of both convergence speed and solution quality, is highly impacted by the size of the subproblem being solved, denoted as Γ. Fig. 3 visually illustrates this parameter's effect, showing the convergence speed of the average solution quality over ten independent runs of the MFSS for various Γ values. As the figure demonstrates, a higher Γ value leads to faster convergence and improved solution quality until a certain threshold. In practice, Γ should not exceed such a size of the subproblem for which the IPS can find near-optimal solutions within a short computational time, related to τ, and should have the largest possible value for which this is true. As it can be seen in Fig. 3, if the value Γ exceeds such a threshold, the IPS cannot find high-quality solutions for the subproblem and consequently there is a drastic decrease in the performance of the MFSS. In the case of the conducted computational experiments, performed on the previously mentioned computer, the best results are acquired for the value $\Gamma = 1000$. It is important to point out that the value of Γ is hardware-dependent, and should be tuned depending on it.

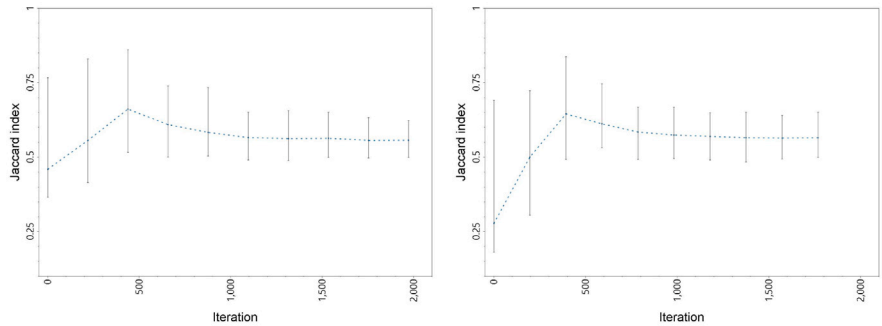
Next, we turn our attention to the parameter n, which controls the number of best solutions considered when generating the base solution B and the set of test solutions S_{kn}. The MFSS aims to tackle larger problem instances than those that can be solved using the IP alone, and the impact of n on the method's performance is particularly relevant in this context. Specifically, we focus on scenarios where the number of variables in the IP is much larger than Γ, i.e., the maximum subproblem size for which the IPS can be efficiently applied.

To investigate the effect of n, we conduct a series of experiments on a problem instance with 1000 items of LK type, using $\Gamma = 250$ and averaging the results of ten independent runs of the MFSS. Fig. 4 presents the average convergence speed as a function of n. Note that for this test instance the corresponding IP has 7000 decision variables which is 28 times larger than the number of decision variables in the subproblem being solved. As the figure shows, a smaller value of n leads to a higher initial convergence speed but also increases the risk of getting trapped in a local optimum. In contrast, a larger value of n allows the MFSS to search a wider solution space and escape local optima more easily, at the cost of slower convergence.

In practice, the choice of n should depend on the available execution time and the desired solution quality. If there is sufficient time, using a larger value of n can yield better results. However, if time is limited or a good solution is already found, a smaller value of n may suffice. Overall, the value of n should be selected carefully based on the specific problem instance and requirements.

Let us make a few notes about the behavior of the IPS time limit τ during the execution of the MFSS. It has been empirically observed that in the case of the computational experiments corresponding to Tables 1–3 stagnation would occur relatively frequently and the value of τ was generally doubled 9 to 10 times. The vast majority of best solution improvements has been achieved for a value of τ up to 0.8 s. On the other hand, for larger values of τ, the IPS for the subproblem would be solved to proven optimality in a much lower time than the time limit. This is to a large extent a consequence of the IP for the subproblem being highly constraint in addition to having a high-quality incumbent solution. In the case of large values of τ, the vast majority of computational time was "wasted" on a few hard-to-solve IPs. This indicates that in future extensions of the MFSS, it is reasonable to increase the value of Γ, size of the subproblem being solved, with the increase of τ above a certain threshold.

To finalize this subsection, let us make an observation that the MFSS's performance relies significantly on the chosen IPS. Specifically,



(a) Instance 6 of type O with 1000 items (b) Instance 6 of type L with 1000 items

Fig. 5. The average, maximal and minimal values of the Jaccard index of pairs of fixed sets generated in $l = 200$ consecutive iterations, starting from iteration s, of the MFSS. The results are provided for two representative instances.

it hinges on the scale of problems the IPS can effectively handle. In case of the KPF, we have empirically observed that the MFSS excels in addressing problem instances ranging from 7 to 17 times larger than those effectively managed by the IPS. In contrast, standard metaheuristics often prove applicable to substantially larger problem instances than the IPS. In practical terms, this implies that beyond a certain problem size, utilizing metaheuristics is more suitable than employing the MFSS.

4.4. Fixed set diversity

The aim of the last group of computational experiments is to evaluate the diversity of fixed sets generated during the execution of the MFSS. This aspect of the MFSS is examined since it can provide some insight in the potential of the MFSS to avoid being trapped in locally optimal solutions by having the potential capability of generating new ones.

One reason for the MFSS getting trapped in such locally optimal solutions is that it generates the same fixed sets or very similar ones. To evaluate the diversity of the generated fixed sets, the first measure is based on the Jaccard index (Jaccard, 1912), also known as the Jaccard similarity coefficient, that measures the similarity between two sets. Formally, the Jaccard index for sets A and B has the following form.

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} \tag{23}$$

In Eq. (23), the notation $J(A, B)$ is used for the Jaccard index of sets A and B. It is equal to the cardinality of the intersection of sets A and B divided by the cardinality of the union of the same two sets. Note that $0 \leq J(A, B) \leq 1$, where the value of 1 indicates the sets are the same while the value of 0 means there is no similarity between them.

This measure is used to evaluate the diversity of fixed sets in the following way. Since it is expected that the diversity of fixed sets will change over the execution of the MFSS, the similarity is evaluated for fixed sets generated over consecutive iterations of the algorithm. To provide greater clarity, the Jaccard index is computed for every pair of fixed sets generated between iterations s and $s + l$ (for an appropriate value of interest l). Fig. 5 illustrates the average, maximum, and minimum Jaccard index values over 200 consecutive iterations ($l = 200$) for two instances comprising 1000 items of types O and LK. It is important to highlight that in Fig. 5, the value s represents the iteration number. It should be noted that a similar behavior has been observed for other instances. From these results, it can be deduced that the similarity of the fixed sets increases as the algorithm progresses and stabilizes with an average Jaccard index of around 0.55. This indicates that even in the later iterations of the algorithm the generated fixed sets maintain diversity.

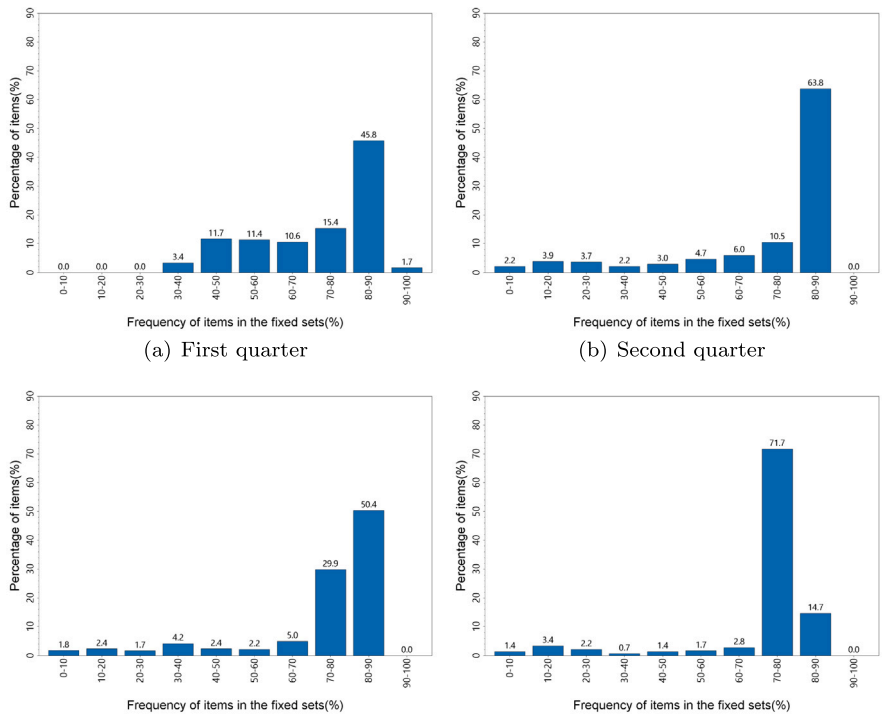
A second reason for the MFSS being easily trapped in locally optimal solutions is that the fixed sets repeatedly contain the same group of items. This type of behavior cannot be recognized using only the Jaccard index. Because of this the frequency of items occurring in the fixed sets is also evaluated. To be more precise, the number of times a value of a variable related to an item is fixed in the IP is also evaluated (this may be interpreted as a frequency-based memory; see, e.g., Glover and Kochenberger (1996)). A graphical illustration of the frequency an item i occurs in a fixed set either as $(i, \top)$ or $(i, \bot)$ is presented in Fig. 6. To be specific, the figure provides the information on the percentage of items that appear in fixed sets within a specific frequency (number of fixed sets out of the total). This distribution is given separately for the first, second, third and fourth quarter of the iterations. It should be noted that a similar behavior, of the instance presented in Fig. 6, can be observed for other instances. The most important observations that can be made from these results is that except in the first quarter of the iterations there are no items for which the value of the corresponding variable in the IP is always fixed. This is a good indicator that MFSS has a high potential of generating new solutions even in the late stages of the algorithm. An interesting effect of the method for diversification of the fixed sets is that in the later iterations the number of items which are a part of 80%–90% of the fixed sets decreases.

5. Conclusions

In this study, a matheuristic approach for solving the KPF based on the FSS algorithm is proposed. The method introduces a novel ground set of elements for the KPF that increases the amount of information provided by the fixed set, and adapts the method for generating fixed sets to increase solution diversity. One of the main advantages of the proposed approach is its simplicity of implementation, as it does not require defining solution neighborhoods and uses the FSS learning mechanism. Computational experiments show that the proposed method significantly outperforms the state-of-the-art approaches for the KPF and is effective for a wide range of method parameters.

It is worth noting that the proposed approach does not significantly rely on any specific properties of the KPF, but rather only uses its IP model. Note that to adapt the MFSS procedure to other problems there are two key points that are problem-dependent and that require knowledge of the domain of the problem at hand, namely the generation of the initial population of solutions and the efficient estimation of the computational cost of the subproblem being solved, once some variables have been fixed based on the fixed set.

Therefore, the approach has the potential for being applied to other 0–1 problems, such as the minimum vertex cover problem, the facility location problem and the set covering problem. Moreover, the simplicity of the approach allows for potential improvements in performance through hybridization with other heuristic/metaheuristic approaches or by exploiting specific properties of the KPF. Another direction of future research could be the extension of the MFSS from problems with binary variables to problems using integer ones.



(a) First quarter (b) Second quarter

(c) Third quarter (d) Fourth quarter

Fig. 6. Graphical illustration of the percentage of items that appear in generated fixed sets within a specific frequency (number of fixed sets in which an item occurs compared to the total) given in percentage during the execution of the algorithm. The values represent the average of these values over ten runs of the MFSS for instance 6 with 1000 items of type LK. A separate subfigure is given for this distribution for the fixed sets generated in the first, second, third and fourth quarter of iterations of the algorithm.

CRediT authorship contribution statement

Raka Jovanovic: Writing – original draft, Visualization, Validation, Software, Methodology, Conceptualization. **Stefan Voß:** Writing – review & editing, Methodology, Conceptualization.

Data availability

Data will be made available on request.

References

Angelelli, E., Mansini, R., Speranza, M.G., 2010. Kernel search: A general heuristic for the multi-dimensional knapsack problem. Comput. Oper. Res. 37 (11), 2017–2026.

Basnet, C., 2018. Heuristics for the multiple knapsack problem with conflicts. Int. J. Oper. Res. 32 (4), 514–525.

Bettinelli, A., Cacchiani, V., Malaguti, E., 2017. A branch-and-bound algorithm for the knapsack problem with conflict graph. INFORMS J. Comput. 29 (3), 457–473.

Boschetti, M.A., Maniezzo, V., 2022. Matheuristics: using mathematics for heuristic design. 4OR 20 (2), 173–208.

Capobianco, G., D'Ambrosio, C., Pavone, L., Raiconi, A., Vitale, G., Sebastiano, F., 2022. A hybrid metaheuristic for the knapsack problem with forfeits. Soft Comput. 26, 749–762. http://dx.doi.org/10.1007/s00500-021-06331-x.

Caserta, M., Voß, S., 2013. A math-heuristic Dantzig–Wolfe algorithm for capacitated lot sizing. Ann. Math. Artif. Intell. 69, 207–224. http://dx.doi.org/10.1007/s10472-013-9339-9.

Cerrone, C., Cerulli, R., Golden, B., 2017. Carousel greedy: A generalized greedy algorithm with applications in optimization. Comput. Oper. Res. 85, 97–112.

Cerulli, R., D'Ambrosio, C., Raiconi, A., Vitale, G., 2020. The knapsack problem with forfeits. In: Combinatorial Optimization: 6th International Symposium, ISCO 2020, Montreal, QC, Canada, May, 2020. 4–6, Revised Selected Papers. Springer, Berlin, Heidelberg, pp. 263–272. http://dx.doi.org/10.1007/978-3-030-53262-8_22.

Chen, Y., Wang, F., Ma, Y., Yao, Y., 2020. A distributed framework for solving and benchmarking security constrained unit commitment with warm start. IEEE Trans. Power Syst. 35 (1), 711–720. http://dx.doi.org/10.1109/TPWRS.2019.2930706.

Coniglio, S., Furini, F., San Segundo, P., 2021. A new combinatorial branch-and-bound algorithm for the knapsack problem with conflicts. European J. Oper. Res. 289 (2), 435–455.

Cornet, A., Laforest, C., 2018. Domination problems with no conflicts. Discrete Appl. Math. 244, 78–88. http://dx.doi.org/10.1016/j.dam.2018.03.018.

Dahmani, I., Hifi, M., 2021. A modified descent method-based heuristic for binary quadratic knapsack problems with conflict graphs. Ann. Oper. Res. 298, 125–147.

D'Ambrosio, C., Laureana, F., Raiconi, A., Vitale, G., 2023. The knapsack problem with forfeit sets. Comput. Oper. Res. 151, 106, 093.

Danna, E., Rothberg, E., Pape, C., 2005. Exploring relaxation induced neighborhoods to improve MIP solutions. Math. Program. 102, 71–90. http://dx.doi.org/10.1007/s10107-004-0518-7.

Darmann, A., Pferschy, U., Schauer, J., 2009. Determining a minimum spanning tree with disjunctive constraints. In: Algorithmic Decision Theory: First International Conference, ADT 2009, Venice, Italy, October, 2009. 20-23. Proceedings 1. Springer, pp. 414–423.

de Abreu, A.P., Fuchigami, H.Y., 2022. An efficiency and robustness analysis of warm-start mathematical models for idle and waiting times optimization in the flow shop. Comput. Ind. Eng. 166, 107976. http://dx.doi.org/10.1016/j.cie.2022.107976.

Dror, M., Orlin, J.B., 2008. Combinatorial optimization with explicit delineation of the ground set by a collection of subsets. SIAM J. Discrete Math. 21 (4), 1019–1034.

Feo, T.A., Resende, M.G., 1995. Greedy randomized adaptive search procedures. J. Global Optim. 6 (2), 109–133.

Galli, L., Martello, S., Rey, C., Toth, P., 2023. Lagrangian matheuristics for the quadratic multiple knapsack problem. Discrete Appl. Math. 335, 36–51.

Gendreau, M., Laporte, G., Semet, F., 2004. Heuristics and lower bounds for the bin packing problem with conflicts. Comput. Oper. Res. 31 (3), 347–358.

Glover, F., Kochenberger, G.A., 1996. Critical event tabu search for multidimensional knapsack problems. In: Osman, I.H., Kelly, J.P. (Eds.), Meta-Heuristics: Theory and Applications. Kluwer, Boston, pp. 407–427. http://dx.doi.org/10.1007/978-1-4613-1361-8_25.

Hanafi, S., Wilbaut, C., 2008. Scatter search for the 0–1 multidimensional knapsack problem. J. Math. Model. Algorithms 7, 143–159. http://dx.doi.org/10.1007/s10852-008-9078-9.

Hifi, M., Otmani, N., 2012. An algorithm for the disjunctively constrained knapsack problem. Int. J. Oper. Res. 13 (1), 22–43.

Hill, A., Baldacci, R., Voß, S., 2022. Optimal Steiner trees under node and edge privacy conflicts. J. Comb. Optim. 43, 1509–1533. http://dx.doi.org/10.1007/s10878-020-00690-1.

Jaccard, P., 1912. The distribution of the flora in the alpine zone. New Phytol. 11 (2), 37–50.

Jovanovic, R., Sanfilippo, A.P., Voß, S., 2022. Fixed set search applied to the multi-objective minimum weighted vertex cover problem. J. Heuristics 28, 481–508. http://dx.doi.org/10.1007/s10732-022-09499-z.

Jovanovic, R., Sanfilippo, A.P., Voß, S., 2023. Fixed set search applied to the clique partitioning problem. European J. Oper. Res. 309, 65–81. http://dx.doi.org/10.1016/j.ejor.2023.01.044.

Jovanovic, R., Tuba, M., Voß, S., 2019. Fixed set search applied to the traveling salesman problem. In: International Workshop on Hybrid Metaheuristics. Springer, pp. 63–77. http://dx.doi.org/10.1007/978-3-030-05983-5_5.

Jovanovic, R., Voß, S., 2019. Fixed set search applied to the minimum weighted vertex cover problem. In: International Symposium on Experimental Algorithms. Springer, pp. 490–504. http://dx.doi.org/10.1007/978-3-030-34029-2_31.

Jovanovic, R., Voss, S., 2020. The fixed set search applied to the power dominating set problem. Expert Syst. 37 (6), e12, 559.

Jovanovic, R., Voß, S., 2021. Fixed set search application for minimizing the makespan on unrelated parallel machines with sequence-dependent setup times. Appl. Soft Comput. 110, 107, 521.

Jovanovic, R., Voß, S., 2024. Matheuristic fixed set search applied to the multidimensional knapsack problem and the knapsack problem with forfeit sets. OR Spectrum 1–37.

Lozano-Osorio, I., Sánchez-Oro, J., Martínez-Gavara, A., López-Sánchez, A.D., Duarte, A., 2023. An efficient fixed set search for the covering location with interconnected facilities problem. In: Metaheuristics: 14th International Conference. Springer, pp. 485–490. http://dx.doi.org/10.1007/978-3-031-26504-4_37.

Maniezzo, V., Boschetti, M.A., Stützle, T., 2021. Kernel search. In: Matheuristics. Springer, pp. 189–197.

Muritiba, A.E.F., Iori, M., Malaguti, E., Toth, P., 2010. Algorithms for the bin packing problem with conflicts. INFORMS J. Comput. 22 (3), 401–415.

Nemhauser, G., Wolsey, L., 1988. The Scope of Integer and Combinatorial Optimization. Vol. 1, John Wiley & Sons, pp. 1–26, chap I..

PassMark, 2022. CPU benchmark. URL www.cpubenchmark.net, last visited 2022-01-30.

Pferschy, U., Schauer, J., 2017. Approximation of knapsack problems with conflict and forcing graphs. J. Comb. Optim. 33 (4), 1300–1323.

Pour, S.M., Drake, J.H., Ejlertsen, L.S., Rasmussen, K.M., Burke, E.K., 2018. A hybrid constraint programming/mixed integer programming framework for the preventive signaling maintenance crew scheduling problem. European J. Oper. Res. 269, 341–352. http://dx.doi.org/10.1016/j.ejor.2017.08.033.

Rosing, K., ReVelle, C., 1997. Heuristic concentration: Two stage solution construction. European J. Oper. Res. 97 (1), 75–86.

Salem, M.Ben., Taktak, R., Mahjoub, A.R., Ben-Abdallah, H., 2018. Optimization algorithms for the disjunctively constrained knapsack problem. Soft Comput. 22, 2025–2043.

Sarhani, M., Voß, S., Jovanovic, R., 2023. Initialization of metaheuristics: comprehensive review, critical analysis, and research directions. Int. Trans. Oper. Res. 30, 3361–3397. http://dx.doi.org/10.1111/itor.13237.

Sniedovich, M., Voß, S., 2006. The corridor method: A dynamic programming inspired metaheuristic. Cybernetics 35 (3), 551–578.

Sondergeld, L., Voß, S., 1999. Cooperative intelligent search using adaptive memory techniques. In: Voß, S., Martello, S., Osman, I., Roucairol, C. (Eds.), Meta-Heuristics: Advances and Trends in Local Search Paradigms for Optimization. Kluwer, Boston, pp. 297–312. http://dx.doi.org/10.1007/978-1-4615-5775-3_21.

Taillard, E., Voß, S., 2002. POPMUSIC – A partial optimization metaheuristic under special intensification conditions. In: Ribeiro, C., Hansen, P. (Eds.), Essays and Surveys in Metaheuristics. Kluwer, Boston, pp. 613–629. http://dx.doi.org/10.1007/978-1-4615-1507-4_27.

Voß, S., Gutenschwager, K., 1998. A chunking based genetic algorithm for the Steiner tree problem in graphs. In: Pardalos, P., Du, D.Z. (Eds.), Network Design: Connectivity and Facilities Location. In: DIMACS Series in Discrete Mathematics and Theoretical Computer Science, vol. 40, AMS, Princeton, pp. 335–355.

Wang, Y., Lü, Z., Glover, F., Hao, J.K., 2013. Probabilistic GRASP-tabu search algorithms for the UBQP problem. Comput. Oper. Res. 40, 3100–3107. http://dx.doi.org/10.1016/j.cor.2011.12.006.

Woodruff, D., 1998. Proposals for chunking and tabu search. European J. Oper. Res. 106, 585–598.

Yamada, T., Kataoka, S., Watanabe, K., 2002. Heuristic and exact algorithms for the disjunctively constrained knapsack problem. Inf. Process. Soc. Japan J. 43 (9), 2864–2870.